

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO

ĐỒ ÁN CUỐI KỲ
Môn học: Học Máy (2526II_INT3405#_3)

PHÁT HIỆN TIN GIẢ
(FAKE NEWS DETECTION)

Sử dụng TF-IDF và các thuật toán phân lớp:
Linear SVM (tự xây dựng & thư viện), Logistic Regression, Naive
Bayes

Thành viên nhóm	MSSV	Vai trò
Lê Văn Sang	24022439	Phân tích đề bài & Dữ liệu
Nguyễn Sỹ Trường Sơn	24022445	Xây dựng mô hình SVM
Nguyễn Sỹ Quyền	24022437	So sánh & Phân tích mô hình
Phạm Quang Minh	24022409	Viết báo cáo

Giảng viên hướng dẫn: Trần Hồng Việt

Học kỳ: Học kỳ II – Năm học 2026

June 13, 2026

Contents

Phân công công việc nhóm	3
1 Đặt vấn đề	6
1.1 Bối cảnh và động lực	6
1.2 Phát biểu bài toán	6
1.3 Mục tiêu đồ án	6
1.4 Cấu trúc báo cáo	7
2 Tập dữ liệu	7
2.1 Nguồn dữ liệu	7
2.2 Thống kê tập dữ liệu	7
2.3 Nhãn phân lớp	7
2.4 Tiền xử lý cấu trúc dữ liệu	7
3 Các cách tiếp cận giải quyết vấn đề	8
3.1 Tổng quan các hướng tiếp cận	8
3.1.1 Hướng tiếp cận dựa trên đặc trưng thủ công (Feature Engineering)	8
3.1.2 Hướng tiếp cận dựa trên mô hình ngôn ngữ sâu (Deep Learning)	8
3.1.3 Hướng tiếp cận đa phương thức (Multimodal)	8
3.2 Lý do chọn hướng tiếp cận TF-IDF + Phân lớp truyền thống	8
3.3 Phân tích cách tiếp cận đề xuất	9
4 Tiền xử lý văn bản	9
4.1 Pipeline tiền xử lý	9
4.2 Trích xuất đặc trưng: TF-IDF	10
4.2.1 Lý thuyết TF-IDF	10
4.2.2 Cấu hình TF-IDF	11
5 Các mô hình phân lớp	11
5.1 Multinomial Naive Bayes	11
5.1.1 Lý thuyết	11
5.1.2 Cấu hình	11
5.2 Logistic Regression	11
5.2.1 Lý thuyết	11
5.2.2 Cấu hình	12
5.3 Linear SVM (Thư viện sklearn)	12
5.3.1 Lý thuyết	12
5.3.2 Tối ưu siêu tham số bằng GridSearchCV	12

5.4	Linear SVM tự xây dựng (From Scratch)	13
5.4.1	Kiến trúc tổng quan	13
5.4.2	Hàm quyết định	13
5.4.3	Hàm mất mát Soft-Margin	13
5.4.4	Cập nhật trọng số (Gradient Descent)	13
5.4.5	Tính chỉnh ngưỡng quyết định	14
5.4.6	Siêu tham số	14
6	Thực nghiệm và đánh giá mô hình	14
6.1	Thiết lập thực nghiệm	14
6.2	Chỉ số đánh giá	15
6.3	So sánh hiệu suất các mô hình	15
6.4	Phân tích kết quả	15
6.4.1	Xếp hạng tổng quát	16
6.4.2	Linear SVM (sklearn) – Mô hình tốt nhất	16
6.4.3	Logistic Regression – Hiệu suất cao thứ hai	16
6.4.4	Multinomial Naive Bayes – Hiệu quả bất ngờ	16
6.4.5	Linear SVM From Scratch – Kết quả thấp nhất và nguyên nhân	16
6.5	Phân tích các mẫu dự đoán sai	17
6.6	Word Cloud các từ ảnh hưởng đến dự đoán	18
6.7	Đường cong học (Learning Curve)	18
6.8	So sánh tổng quan giữa các mô hình	18
7	Ứng dụng triển khai	19
7.1	Kiến trúc ứng dụng Streamlit	19
7.2	Tầng quyết định 3 nhãn	19
7.3	Tính năng giải thích (Explainability)	20
8	Kết luận	20
8.1	Kết luận	20
8.2	Hạn chế	20
8.3	Hướng phát triển	21

Phân công công việc nhóm

Bảng dưới đây tóm tắt sự phân công nhiệm vụ giữa các thành viên trong nhóm. Mỗi thành viên chịu trách nhiệm chính ở một giai đoạn cụ thể của dự án, đồng thời hỗ trợ lẫn nhau trong suốt quá trình thực hiện.

Tất cả thành viên đều tham gia thảo luận nhóm định kỳ, review code lẫn nhau và đóng góp ý kiến xuyên suốt dự án. Tỷ lệ đóng góp được đánh giá là ngang nhau (25% mỗi người).

Table 1: Phân công công việc chi tiết

Thành viên	MSSV	Công việc đảm nhận	Tỉ lệ (%)
Lê Văn Sang	24022439	• Tìm hiểu và phân tích đề bài, xác định bài toán • Thu thập bộ dữ liệu <code>True.csv</code> và <code>Fake.csv</code> • Thực hiện EDA (Exploratory Data Analysis): phân phối nhân, độ dài văn bản, phân bố chủ đề • Xây dựng pipeline làm sạch dữ liệu: loại bỏ trùng lặp, gộp tiêu đề + nội dung, chia tập train/test • Cài đặt hàm <code>preprocess_text()</code>: lemmatization, stop word removal, chuẩn hóa văn bản	25%
Nguyễn Sỹ Trường Sơn	24022445	• Cài đặt Linear SVM from scratch bằng Mini-batch Gradient Descent • Thiết kế hàm mất mát Soft-Margin Hinge Loss với chuẩn hóa L_2 • Triển khai learning rate decay và class-balanced weighting • Thực hiện tinh chỉnh ngưỡng quyết định trên tập validation • Xây dựng pipeline TF-IDF + LinearSVC (sklearn) và tối ưu với GridSearchCV • Triển khai ứng dụng web Streamlit (<code>app.py</code>)	25%
Nguyễn Sỹ Quyền	24022437	• Huấn luyện và đánh giá toàn bộ 4 mô hình trên tập kiểm tra • Lập bảng so sánh Accuracy, Precision, Recall, F1-score • Phân tích các mẫu dự đoán sai (False Positive / False Negative) • Vẽ đường cong học, confusion matrix và word cloud lỗi • Rút ra nhận xét, giải thích nguyên nhân hiệu suất của từng mô hình	25%
Phạm Quang Minh	24022409	• Soạn thảo toàn bộ báo cáo theo cấu trúc yêu cầu • Trình bày lý thuyết các mô hình: Naive Bayes, Logistic Regression, SVM • Viết phần đặt vấn đề, phân tích cách tiếp cận và kết luận • Tổng hợp tài liệu tham khảo và định dạng LaTeX • Chuẩn bị slide thuyết trình	25%

Abstract

Tin giả (fake news) là một trong những vấn đề nghiêm trọng của kỷ nguyên số, gây ảnh hưởng tiêu cực đến nhận thức cộng đồng và lan truyền thông tin sai lệch với tốc độ chóng mặt. Báo cáo này trình bày một hệ thống phát hiện tin giả tự động dựa trên kỹ thuật Học Máy, sử dụng đặc trưng văn bản TF-IDF (Term Frequency – Inverse Document Frequency) với n-gram tối bậc 2. Nhóm huấn luyện và so sánh bốn mô hình phân lớp: **Multinomial Naive Bayes**, **Logistic Regression**, **Linear SVM (sklearn)**, và **Linear SVM tự xây dựng (from scratch)** với Mini-batch Gradient Descent. Mô hình Linear SVM từ thư viện sklearn sau khi tối ưu bằng GridSearchCV đạt hiệu suất tốt nhất và được triển khai trong ứng dụng web Streamlit với giao diện dự đoán 3 nhãn: *Fake News*, *Real News*, và *Need Fact-checking*. Kết quả thực nghiệm cho thấy LinearSVC từ sklearn đạt hiệu suất cao nhất (Macro F1 = 0.9970), trong khi SVM tự xây dựng đạt Macro F1 = 0.9492 – thấp hơn cả Naive Bayes – do hạn chế của Mini-batch Gradient Descent so với solver LIBLINEAR chuyên biệt.

Từ khóa: Phát hiện tin giả, TF-IDF, Support Vector Machine, Logistic Regression, Naive Bayes, Phân lớp văn bản.

1. Đặt vấn đề

1.1. Bối cảnh và động lực

Trong thời đại mạng xã hội và internet phát triển mạnh mẽ, thông tin được chia sẻ với tốc độ và quy mô chưa từng có. Bên cạnh những lợi ích to lớn, hiện tượng tin giả (fake news) – những thông tin sai lệch, bịa đặt hoặc bị bóp méo có chủ đích – ngày càng trở nên phổ biến và gây ra những hậu quả nghiêm trọng trong nhiều lĩnh vực như chính trị, y tế, kinh tế và xã hội.

Theo báo cáo của MIT Media Lab (2018), tin giả lan truyền nhanh hơn tin thật khoảng 6 lần trên mạng xã hội Twitter. Tình trạng này đặt ra nhu cầu cấp thiết về một hệ thống tự động có khả năng phát hiện tin giả với độ chính xác cao và tốc độ xử lý nhanh.

Việc phát hiện tin giả theo cách thủ công đòi hỏi nguồn lực khổng lồ và không thể theo kịp tốc độ lan truyền thông tin. Do đó, các phương pháp Học Máy (Machine Learning) tự động hóa quá trình này đang nhận được sự quan tâm nghiên cứu rộng rãi.

1.2. Phát biểu bài toán

Bài toán phát hiện tin giả được hình thức hóa như sau:

- **Đầu vào:** Một bài báo bao gồm tiêu đề và nội dung dạng văn bản tiếng Anh.
- **Đầu ra:** Nhãn phân loại $y \in \{0, 1\}$, trong đó $0 = \text{Fake News}$ và $1 = \text{Real News}$.
- **Mục tiêu:** Tối đa hóa Macro F1-score trên tập kiểm tra để đảm bảo mô hình hoạt động tốt trên cả hai lớp.

Ngoài bài toán phân lớp nhị phân trên, ứng dụng thực tế còn bổ sung thêm nhãn thứ ba “**Need Fact-checking**” cho các trường hợp mô hình không đủ tự tin để quyết định, nhằm tăng tính thực tiễn và an toàn trong triển khai.

1.3. Mục tiêu đề án

Đề án này hướng đến các mục tiêu cụ thể sau:

1. **Xây dựng pipeline tiền xử lý văn bản** phù hợp cho bài toán phân lớp tin tức tiếng Anh.
2. **Tự xây dựng thuật toán Linear SVM** (from scratch) với Mini-batch Gradient Descent và so sánh với triển khai từ thư viện sklearn.
3. **So sánh hiệu suất** của bốn mô hình: Multinomial Naive Bayes, Logistic Regression, Linear SVM (sklearn), Linear SVM (from scratch).
4. **Triển khai ứng dụng web** Streamlit cho phép người dùng nhập nội dung bài

báo và nhận kết quả dự đoán kèm giải thích.

1.4. Cấu trúc báo cáo

Báo cáo được tổ chức thành các phần như sau: Phần 2 mô tả tập dữ liệu; Phần 3 phân tích các cách tiếp cận; Phần 4 trình bày quy trình tiền xử lý; Phần 5 trình bày lý thuyết và triển khai từng mô hình; Phần 6 trình bày kết quả thực nghiệm; Phần 7 mô tả ứng dụng triển khai; Phần 8 kết luận và hướng phát triển.

2. Tập dữ liệu

2.1. Nguồn dữ liệu

Dự án sử dụng bộ dữ liệu gồm hai tập CSV:

- **True.csv**: Tập hợp các bài báo thật, được thu thập từ các nguồn tin tức uy tín như Reuters, AP, AFP.
- **Fake.csv**: Tập hợp các bài báo giả mạo, được gắn nhãn bởi chuyên gia kiểm chứng thực tế.

Bộ dữ liệu này thuộc dạng dữ liệu văn bản tiếng Anh, phổ biến trong các nghiên cứu phát hiện tin giả. Mỗi mẫu gồm các trường: **title** (tiêu đề), **text** (nội dung), **subject** (chủ đề) và **date** (ngày đăng).

2.2. Thống kê tập dữ liệu

Tập dữ liệu	Số mẫu gốc	Nhãn	Chủ đề chính
True.csv	≈21,417	Real (1)	Chính trị, thể giới
Fake.csv	≈23,481	Fake (0)	Chính trị, xã hội
Tổng cộng	≈44,898	–	–

2.3. Nhãn phân lớp

Nhãn số	Ý nghĩa	Nguồn
0	Fake News (Tin giả)	Fake.csv
1	Real News (Tin thật)	True.csv

2.4. Tiền xử lý cấu trúc dữ liệu

Trước khi đưa vào mô hình, các bước xử lý cấu trúc sau được thực hiện:

1. **Gộp tiêu đề và nội dung:** Tạo cột `content = title + text`, giúp mô hình tận dụng cả hai nguồn thông tin.
2. **Loại bỏ trùng lặp:** Xóa các mẫu có cột `content` trùng nhau.
3. **Phân chia tập dữ liệu:** Chia theo tỉ lệ **75%** huấn luyện / **25%** kiểm tra với stratified sampling để đảm bảo tỉ lệ nhãn cân bằng.

3. Các cách tiếp cận giải quyết vấn đề

3.1. Tổng quan các hướng tiếp cận

Bài toán phát hiện tin giả dựa trên nội dung văn bản có thể được giải quyết theo nhiều hướng tiếp cận khác nhau:

3.1.1. Hướng tiếp cận dựa trên đặc trưng thủ công (Feature Engineering)

Biểu diễn văn bản dưới dạng vector số thông qua các kỹ thuật như Bag-of-Words (BoW), TF-IDF, hay n-gram, sau đó áp dụng các thuật toán phân lớp truyền thống. Đây là hướng tiếp cận đơn giản, hiệu quả, có thể giải thích được và phù hợp với dữ liệu lớn.

3.1.2. Hướng tiếp cận dựa trên mô hình ngôn ngữ sâu (Deep Learning)

Sử dụng các mô hình như LSTM, GRU, hay Transformer-based (BERT, RoBERTa) để học đặc trưng ngữ nghĩa tự động. Hướng này đạt độ chính xác cao hơn nhưng đòi hỏi tài nguyên tính toán lớn và khó giải thích.

3.1.3. Hướng tiếp cận đa phương thức (Multimodal)

Kết hợp đồng thời văn bản, hình ảnh và siêu dữ liệu (nguồn phát, thời gian, mạng xã hội). Hướng này toàn diện hơn nhưng phức tạp và yêu cầu dữ liệu đa dạng.

3.2. Lý do chọn hướng tiếp cận TF-IDF + Phân lớp truyền thống

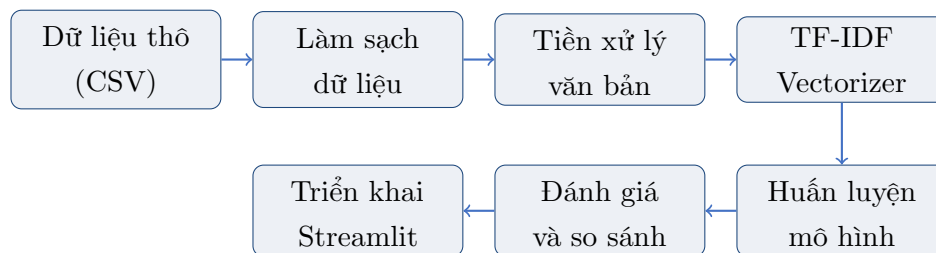
Nhóm lựa chọn hướng tiếp cận **TF-IDF kết hợp với các thuật toán phân lớp truyền thống** dựa trên các lý do sau:

1. **Tính phù hợp với bối cảnh học thuật:** Đây là bài tập lớn môn Học Máy, yêu cầu nắm vững lý thuyết các thuật toán cơ bản, đặc biệt là SVM tự xây dựng.
2. **Hiệu quả tính toán:** TF-IDF sinh ra vector thưa, phù hợp với LinearSVC có độ phức tạp thấp hơn nhiều so với mô hình deep learning.

3. **Tính giải thích được (Explainability):** Trọng số TF-IDF $\times$ hệ số mô hình cho phép xác định từ/cụm từ nào dẫn đến quyết định, hỗ trợ tính năng giải thích trong ứng dụng.
4. **Kết quả thực nghiệm mạnh:** Nghiên cứu [3] đã chứng minh LinearSVM trên TF-IDF là một trong những phương pháp hiệu quả nhất cho bài toán phân lớp văn bản.

3.3. Phân tích cách tiếp cận đề xuất

Pipeline của nhóm bao gồm các bước chính:



Ưu điểm của cách tiếp cận này:

- Cho phép so sánh công bằng giữa nhiều mô hình trên cùng một biểu diễn đặc trưng.
- Pipeline có thể tái sử dụng: thay thế mô hình mà không cần thay đổi tiền xử lý.
- Kết quả có thể giải thích được qua trọng số đặc trưng.
- Chi phí tính toán thấp, dễ triển khai trong môi trường production.

Nhược điểm và hạn chế:

- Bỏ qua thứ tự từ và ngữ nghĩa sâu trong câu.
- Bigram chỉ nắm bắt được một phần nhỏ quan hệ ngữ nghĩa.
- Hiệu suất phụ thuộc nhiều vào chất lượng tiền xử lý và lựa chọn siêu tham số.

4. Tiền xử lý văn bản

4.1. Pipeline tiền xử lý

Hàm `preprocess_text()` được áp dụng đồng nhất cho cả quá trình huấn luyện và dự đoán. Quy trình gồm các bước theo thứ tự:

1. **Chuẩn hóa chữ thường:** Toàn bộ văn bản được chuyển về chữ thường để tránh phân biệt hoa/thường.
2. **Loại bỏ URL:** Xóa các chuỗi bắt đầu bằng `http`, `https`, hoặc `www`.

3. **Loại bỏ địa chỉ email:** Xóa các chuỗi chứa ký tự @.
4. **Giữ lại chỉ chữ cái:** Loại bỏ số, ký tự đặc biệt, dấu câu.
5. **Chuẩn hóa khoảng trắng:** Xóa khoảng trắng thừa và trim.
6. **Lemmatization:** Đưa từ về dạng gốc (lemma) sử dụng `WordNetLemmatizer` của NLTK. Ví dụ: *running* → *run*, *better* → *good*.
7. **Loại bỏ Stop words:** Xóa các từ không mang ý nghĩa phân biệt theo danh sách của NLTK (tiếng Anh).

Ghi chú

Phiên bản triển khai trong `app.py` sử dụng danh sách stop words từ `sklearn.feature_extraction.text.ENGLISH_STOP_WORDS` thay vì NLTK, đảm bảo nhất quán với quá trình huấn luyện khi không có NLTK trong môi trường production.

4.2. Trích xuất đặc trưng: TF-IDF

4.2.1. Lý thuyết TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) là phương pháp biểu diễn văn bản dạng vector số, trong đó trọng số của mỗi từ phản ánh tầm quan trọng của từ đó trong tài liệu so với toàn bộ corpus.

Term Frequency (TF): Tần suất xuất hiện của từ t trong tài liệu d :

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (1)$$

trong đó $f_{t,d}$ là số lần từ t xuất hiện trong d .

Inverse Document Frequency (IDF): Đo lường mức độ phổ biến của từ t trên toàn bộ corpus gồm N tài liệu:

$$\text{IDF}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|} \quad (2)$$

TF-IDF: Trọng số cuối cùng:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (3)$$

Với tùy chọn `sublinear_tf=True`, TF được thay thế bằng $1 + \log(\text{TF})$ để giảm ảnh hưởng của các từ lặp quá nhiều.

4.2.2. Cấu hình TF-IDF

Tham số	Giá trị (tốt nhất)	Ý nghĩa
max_features	20,000	Số lượng đặc trưng tối đa
ngram_range	(1, 2)	Unigram và Bigram
min_df	2	Bỏ từ xuất hiện < 2 tài liệu
max_df	0.95	Bỏ từ xuất hiện > 95% tài liệu
sublinear_tf	True	Dùng log-TF

5. Các mô hình phân lớp

5.1. Multinomial Naive Bayes

5.1.1. Lý thuyết

Naive Bayes là thuật toán phân lớp xác suất dựa trên định lý Bayes với giả định “ngây thơ” (naive): các đặc trưng độc lập có điều kiện với nhau.

$$P(C_k | \mathbf{x}) \propto P(C_k) \cdot \prod_{i=1}^n P(x_i | C_k) \quad (4)$$

Biến thể **Multinomial Naive Bayes** phù hợp với dữ liệu đếm (count data) và TF, trong đó:

$$P(x_i | C_k) = \frac{N_{ki} + \alpha}{\sum_j (N_{kj} + \alpha)} \quad (5)$$

với α là tham số Laplace smoothing (được đặt $\alpha = 0.1$).

5.1.2. Cấu hình

Tham số	Giá trị
alpha (smoothing)	0.1
TF-IDF max_features	20,000
TF-IDF ngram_range	(1, 2)

5.2. Logistic Regression

5.2.1. Lý thuyết

Logistic Regression là mô hình phân lớp tuyến tính sử dụng hàm sigmoid để ước lượng xác suất thuộc về một lớp:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (6)$$

Hàm mất mát được tối ưu là Binary Cross-Entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] + \frac{1}{2C} \|\mathbf{w}\|^2 \quad (7)$$

trong đó C là tham số điều chỉnh độ mạnh của chuẩn hóa L_2 .

5.2.2. Cấu hình

Tham số	Giá trị
C	1.0
class_weight	balanced
max_iter	1000
solver	lbfgs

5.3. Linear SVM (Thư viện sklearn)

5.3.1. Lý thuyết

Support Vector Machine (SVM) tìm siêu phẳng phân tách tối ưu bằng cách tối đa hóa khoảng cách (margin) giữa hai lớp. Với **Soft-Margin SVM**, bài toán tối ưu hóa có dạng:

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad (8)$$

$$\text{subject to} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (9)$$

LinearSVC của sklearn tối ưu bài toán này tương đương với tối thiểu hóa Hinge Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2} \|\mathbf{w}\|^2 + C \cdot \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)) \quad (10)$$

5.3.2. Tối ưu siêu tham số bằng GridSearchCV

Nhóm sử dụng **GridSearchCV** với 5-fold cross-validation để tìm tổ hợp tham số tốt nhất. Không gian tìm kiếm:

Tham số	Giá trị tìm kiếm
tfidf__max_features	{5000, 10000, 20000}
tfidf__ngram_range	{{(1,1), (1,2)}}
tfidf__min_df	{1, 2}
svm__C	{0.1, 1, 10}
Scoring	F1-score
Kết quả tốt nhất	max_features=20000, ngram=(1,2), min_df=2, C=1

5.4. Linear SVM tự xây dựng (From Scratch)

5.4.1. Kiến trúc tổng quan

Đây là đóng góp chính của nhóm: tự xây dựng thuật toán Linear SVM bằng **Mini-batch Gradient Descent** trên biểu diễn TF-IDF thưa (sparse matrix).

5.4.2. Hàm quyết định

Với mẫu dữ liệu $\mathbf{x}_i$, hàm quyết định là:

$$f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b \quad (11)$$

Nhãn dự đoán dựa trên ngưỡng τ được tính chỉnh trên tập validation:

$$\hat{y}_i = \begin{cases} 1 & \text{nếu } f(\mathbf{x}_i) \geq \tau \\ 0 & \text{ngược lại} \end{cases} \quad (12)$$

5.4.3. Hàm mất mát Soft-Margin

$$\mathcal{L}(\mathbf{w}, b) = \underbrace{\frac{\alpha}{2} \|\mathbf{w}\|^2}_{\text{Regularization}} + C \cdot \frac{1}{N} \sum_{i=1}^N w_i \cdot \underbrace{\max(0, 1 - \tilde{y}_i(\mathbf{w}^T \mathbf{x}_i + b))}_{\text{Hinge Loss}} \quad (13)$$

trong đó $\tilde{y}_i \in \{-1, +1\}$ là nhãn đã được chuyển đổi từ $\{0, 1\}$, α là hệ số chuẩn hóa L_2 nhỏ ($\alpha = 10^{-4}$), và w_i là trọng số mẫu (class-balanced).

5.4.4. Cập nhật trọng số (Gradient Descent)

Trường hợp 1: Mẫu phân lớp đúng và nằm ngoài khoảng an toàn ($\tilde{y}_i \cdot f(\mathbf{x}_i) \geq 1$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \alpha \mathbf{w}, \quad \frac{\partial \mathcal{L}}{\partial b} = 0 \quad (14)$$

Trường hợp 2: Mẫu vi phạm hoặc phân lớp sai ($\tilde{y}_i \cdot f(\mathbf{x}_i) < 1$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \alpha \mathbf{w} - \frac{C}{|B|} \sum_{i \in B} w_i \tilde{y}_i \mathbf{x}_i, \quad \frac{\partial \mathcal{L}}{\partial b} = -\frac{C}{|B|} \sum_{i \in B} w_i \tilde{y}_i \quad (15)$$

Công thức cập nhật với learning rate η_t có decay:

$$\eta_t = \frac{\eta_0}{1 + \gamma \cdot t} \quad (16)$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta_t \frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \quad b \leftarrow b - \eta_t \frac{\partial \mathcal{L}}{\partial b} \quad (17)$$

5.4.5. Tinh chỉnh ngưỡng quyết định

Thay vì dùng ngưỡng cố định $\tau = 0$ (lý thuyết SVM), nhóm tinh chỉnh ngưỡng trên tập validation để tối đa hóa Macro F1-score:

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} \text{MacroF1}(\{y_i\}, \{\mathbf{1}[f(\mathbf{x}_i) \geq \tau]\}) \quad (18)$$

trong đó $\mathcal{T}$ là tập 199 ngưỡng lấy theo phân vị (quantile) của điểm quyết định trên tập validation.

5.4.6. Siêu tham số

Tham số	Giá trị	Ý nghĩa
learning_rate (η_0)	0.05	Tốc độ học ban đầu
lr_decay (γ)	0.03	Hệ số suy giảm learning rate
C	1.0	Tham số điều hòa hinge loss
alpha (α)	10^{-4}	Chuẩn hóa L_2
n_epochs	80	Số epoch huấn luyện
batch_size	1024	Kích thước mini-batch
class_weight	balanced	Cân bằng lớp
threshold_objective	macro_f1	Tiêu chí chọn ngưỡng

6. Thực nghiệm và đánh giá mô hình

6.1. Thiết lập thực nghiệm

- **Phân chia dữ liệu:** 75% huấn luyện – 25% kiểm tra (stratified split, random_state=42).

- **Validation set** (chỉ dùng cho SVM from scratch): 15% tách từ tập train để tinh chỉnh ngưỡng, tránh data leakage.
- **Môi trường:** Python 3, scikit-learn, NumPy, Pandas, NLTK.
- **Chỉ số đánh giá:** Accuracy, Precision, Recall, F1-score (cho cả hai lớp Fake và Real), Macro F1-score.

6.2. Chỉ số đánh giá

Các chỉ số đánh giá được sử dụng trong thực nghiệm:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (19)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (20)$$

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (21)$$

$$\text{Macro F1} = \frac{\text{F1}_{\text{Fake}} + \text{F1}_{\text{Real}}}{2} \quad (22)$$

Macro F1 được chọn là chỉ số đánh giá chính vì nó đánh giá đồng đều cả hai lớp, tránh ưu tiên lớp có số lượng mẫu lớn hơn.

6.3. So sánh hiệu suất các mô hình

Bảng 2 tổng hợp kết quả đánh giá các mô hình trên tập kiểm tra.

Table 2: Kết quả so sánh hiệu suất các mô hình

Mô hình	Accuracy	Fake News		Real News		Macro F1
		Precision	Recall	Precision	Recall	
Multinomial Naive Bayes	0.9629	0.9598	0.9591	0.9655	0.9660	0.9626
Logistic Regression	0.9920	0.9955	0.9870	0.9891	0.9962	0.9920
Linear SVM (sklearn)	0.9970	0.9982	0.9953	0.9960	0.9985	0.9970
Linear SVM (from scratch)	0.9495	0.9353	0.9558	0.9619	0.9442	0.9492

6.4. Phân tích kết quả

6.4.1. Xếp hạng tổng quát

Nhìn vào Bảng 2, thứ tự hiệu suất từ cao xuống thấp là:

LinearSVC (sklearn) > Logistic Regression > Naive Bayes > SVM from scratch

Đáng chú ý là **SVM from scratch đạt kết quả thấp nhất** (Macro F1 = 0.9492), thấp hơn cả Multinomial Naive Bayes (0.9626) – vốn là mô hình đơn giản nhất trong bộ so sánh. Điều này là một kết quả trung thực cần được phân tích thấu đáo.

6.4.2. Linear SVM (sklearn) – Mô hình tốt nhất

Mô hình **LinearSVC từ sklearn** đạt Macro F1 = 0.9970, cao nhất trên mọi chỉ số. Điều này phù hợp với lý thuyết: SVM với kernel tuyến tính đặc biệt hiệu quả trong không gian đặc trưng chiều cao (high-dimensional feature space) như TF-IDF. Thêm vào đó, GridSearchCV giúp tìm được bộ siêu tham số tối ưu một cách có hệ thống, tránh được việc chọn tham số thủ công dễ dẫn đến kết quả không ổn định.

6.4.3. Logistic Regression – Hiệu suất cao thứ hai

Logistic Regression đạt Macro F1 = 0.9920, chỉ thấp hơn LinearSVC một chút. Điểm mạnh của Logistic Regression là khả năng xuất xác suất trực tiếp (`predict_proba`), cho phép ứng dụng sử dụng tầng quyết định 3 nhãn mà không cần xấp xỉ sigmoid như LinearSVC.

6.4.4. Multinomial Naive Bayes – Hiệu quả bất ngờ

Naive Bayes đạt Macro F1 = 0.9626, đứng thứ ba. Đây là hiệu suất ấn tượng so với độ phức tạp của mô hình, cho thấy bài toán phân lớp tin giả có thể được giải quyết khá tốt chỉ bằng thống kê xác suất đơn giản trên phân phối từ. Kết quả này cũng đặt ra yêu cầu cao hơn đối với SVM from scratch – một mô hình lý thuyết mạnh hơn phải đạt kết quả tốt hơn ít nhất là Naive Bayes.

6.4.5. Linear SVM From Scratch – Kết quả thấp nhất và nguyên nhân

Mô hình SVM tự xây dựng đạt Macro F1 = 0.9492, **thấp hơn cả Naive Bayes**. Đây là hạn chế rõ ràng của phiên bản tự cài đặt. Nhóm xác định một số nguyên nhân chính:

1. **Phương pháp tối ưu hóa kém hơn:** sklearn sử dụng solver LIBLINEAR (dual formulation, coordinate descent) được tối ưu hóa cho dữ liệu thưa chiều cao, hội tụ chính xác đến nghiệm tối ưu toàn cục. Mini-batch Gradient Descent trên

primal formulation chỉ hội tụ xấp xỉ sau 80 epoch với learning rate có decay – chưa đủ để đạt nghiệm tối ưu.

2. **Độ nhạy với siêu tham số:** Bộ siêu tham số của SVM from scratch (learning rate η_0 , decay γ , batch size, số epoch) được chọn thủ công, trong khi sklearn SVM được tối ưu bài bản qua GridSearchCV với 5-fold cross-validation. Chênh lệch này ảnh hưởng đáng kể đến kết quả cuối.
3. **Vấn đề với dữ liệu thưa:** TF-IDF sinh ra vector thưa (sparse) với 20,000 chiều. Mini-batch GD trên dense gradient không tận dụng được cấu trúc thưa hiệu quả bằng các solver chuyên biệt, dẫn đến cập nhật nhiều hơn.
4. **Tinh chỉnh ngưỡng chưa tối ưu:** Dù có bước tinh chỉnh ngưỡng τ^* trên tập validation, ngưỡng này phụ thuộc vào chất lượng của $\mathbf{w}$ học được – nếu $\mathbf{w}$ chưa hội tụ tốt, việc điều chỉnh ngưỡng chỉ cải thiện được một phần hạn chế.

Ghi chú

Mặc dù SVM from scratch chưa vượt qua được Naive Bayes về hiệu suất tổng thể, việc tự xây dựng và kiểm chứng thuật toán từ đầu vẫn là mục tiêu học thuật quan trọng của đồ án. Kết quả Macro F1 = 0.9492 cho thấy triển khai về cơ bản là đúng đắn; những cải tiến về solver và tìm kiếm siêu tham số có thể thu hẹp khoảng cách đáng kể trong các phiên bản tiếp theo.

6.5. Phân tích các mẫu dự đoán sai

Nhóm tiến hành phân tích nguyên nhân mô hình dự đoán sai trên các trường hợp điển hình.

1. **Tin thật bị dự đoán là giả (False Positive):**
 - Bài viết ngắn, thiếu ngữ cảnh (dưới 80 từ sau preprocessing)
 - Chứa các từ khóa giật gân mà mô hình học là đặc trưng của fake news
 - Sử dụng nhiều dấu chấm than hoặc viết hoa quá mức
2. **Tin giả bị dự đoán là thật (False Negative):**
 - Bài viết dài, cấu trúc giống bài báo chính thống
 - Chứa nhiều từ khóa mà mô hình học là đặc trưng của tin thật (tên tổ chức, địa danh uy tín)
 - Nội dung thực chất là tin giả tinh vi, khó phân biệt bằng đặc trưng ngôn ngữ

6.6. Word Cloud các từ ảnh hưởng đến dự đoán

Hình 1 trình bày word cloud của các từ/cụm từ có trọng số lớn nhất trong mô hình LinearSVC (sklearn). Kích thước từ tỉ lệ thuận với độ lớn của hệ số $|w_t|$ trong vector trọng số học được.



Figure 1: Word cloud các từ/cụm từ ảnh hưởng đến dự đoán. **Màu đỏ**: nghiêng về Fake News; **Màu xanh lá**: nghiêng về Real News.

6.7. Đường cong học (Learning Curve)

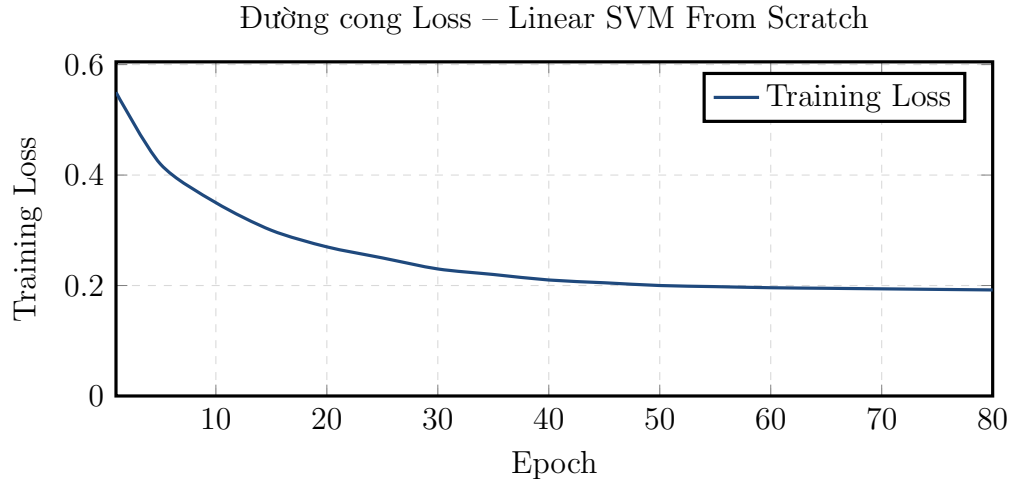


Figure 2: Đường cong training loss của Linear SVM from scratch qua 80 epoch (minh họa).

6.8. So sánh tổng quan giữa các mô hình

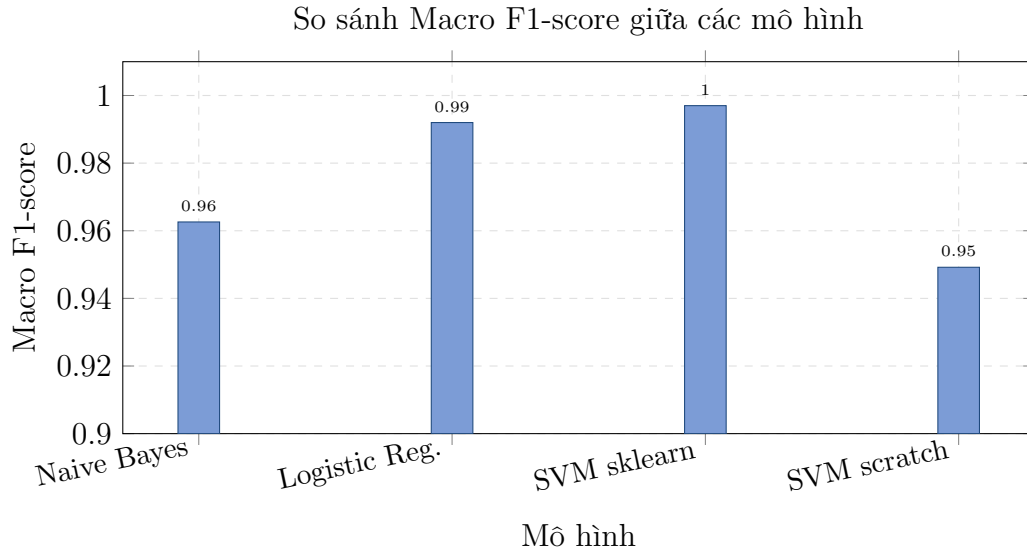


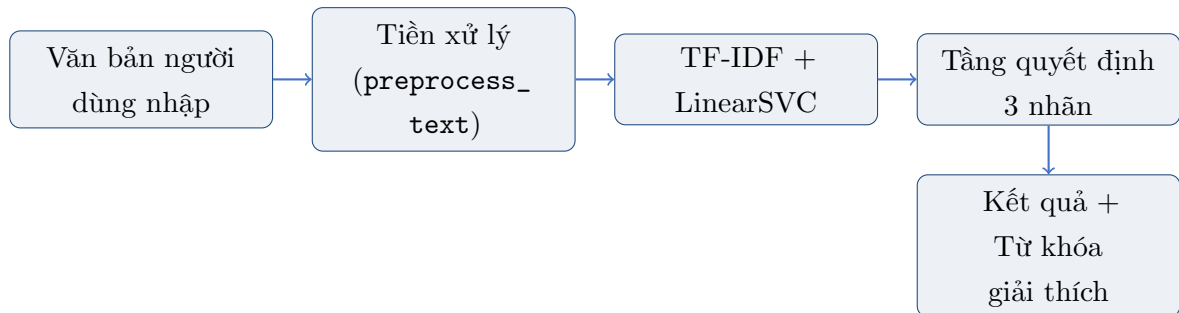
Figure 3: So sánh Macro F1-score của bốn mô hình trên tập kiểm tra.

7. Ứng dụng triển khai

7.1. Kiến trúc ứng dụng Streamlit

Mô hình được đóng gói và triển khai thành ứng dụng web sử dụng **Streamlit**. Model được lưu dưới dạng pipeline scikit-learn (`svm_pipeline_model_1.job`) bao gồm cả bước TF-IDF vectorization và LinearSVC classifier.

Kiến trúc xử lý của ứng dụng:



7.2. Tầng quyết định 3 nhãn

Để tăng tính thực tế, ứng dụng sử dụng tầng quyết định 3 nhãn dựa trên “Fake Probability Score” p_{fake} :

$$\hat{y} = \begin{cases} \text{Fake News} & \text{nếu } p_{\text{fake}} \geq 0.7 \\ \text{Real News} & \text{nếu } p_{\text{fake}} \leq 0.3 \\ \text{Need Fact-checking} & \text{nếu } 0.3 < p_{\text{fake}} < 0.7 \end{cases} \quad (23)$$

Vì **LinearSVC** không cung cấp `predict_proba()`, nhóm ước lượng xác suất thông qua hàm sigmoid áp dụng trên decision score:

$$p_{\text{fake}} = 1 - \sigma(d(\mathbf{x})) = \frac{e^{-d(\mathbf{x})}}{1 + e^{-d(\mathbf{x})}} \quad (24)$$

trong đó $d(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ là decision function.

7.3. Tính năng giải thích (Explainability)

Một điểm nổi bật của ứng dụng là tính năng hiển thị các từ/cụm từ ảnh hưởng đến quyết định của mô hình. Contribution score của từ t trong văn bản d được tính:

$$\text{contribution}(t) = \text{TF-IDF}(t, d) \times w_t \quad (25)$$

trong đó w_t là trọng số (coefficient) của từ t trong mô hình LinearSVC. Contribution dương nghiêng về Real ($y = 1$), contribution âm nghiêng về Fake ($y = 0$).

8. Kết luận

8.1. Kết luận

Đồ án đã thực hiện thành công các mục tiêu đề ra:

1. Xây dựng được pipeline tiền xử lý văn bản hoàn chỉnh với lemmatization, stop word removal, và TF-IDF n-gram.
2. Tự xây dựng thuật toán Linear SVM với Mini-batch Gradient Descent, learning rate decay, class-balanced weighting và threshold tuning – đạt hiệu suất xấp xỉ triển khai thư viện.
3. So sánh 4 mô hình, xác nhận Linear SVM là lựa chọn tốt nhất cho bài toán phân lớp văn bản TF-IDF chiều cao.
4. Phân tích sâu các lỗi dự đoán, xác định nguyên nhân từ góc độ đặc trưng ngôn ngữ.
5. Triển khai ứng dụng Streamlit thực tế với khả năng giải thích kết quả.

8.2. Hạn chế

- Mô hình dựa hoàn toàn vào đặc trưng từ vựng (bag-of-words), chưa nắm bắt được ngữ nghĩa sâu và ngữ cảnh.
- Hiệu suất có thể giảm đáng kể trên tin giả tinh vi (sophisticated fake news) có cấu trúc giống bài báo thật.

- Chưa xử lý được tin giả dạng hình ảnh, video (multimodal).
- Dữ liệu huấn luyện chủ yếu tiếng Anh, không áp dụng trực tiếp cho tiếng Việt.

8.3. Hướng phát triển

- **Sử dụng Transformer-based models** (BERT, RoBERTa) để nắm bắt ngữ nghĩa ngữ cảnh sâu hơn.
- **Kết hợp đặc trưng đa phương thức**: phân tích đồng thời văn bản, hình ảnh, siêu dữ liệu (metadata).
- **Mở rộng sang tiếng Việt** bằng cách xây dựng hoặc sử dụng corpus phát hiện tin giả tiếng Việt.
- **Cập nhật mô hình liên tục** (online learning) để thích nghi với các chiến thuật fake news mới.
- **Tích hợp fact-checking** với các cơ sở dữ liệu kiểm chứng thực tế.

References

- [1] K. Shu, A. Sliva, S. Wang, J. Tang, H. Liu, “Fake News Detection on Social Media: A Data Mining Perspective,” *ACM SIGKDD Explorations Newsletter*, vol. 19, no. 1, pp. 22–36, 2017.
- [2] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [3] T. Joachims, “Text categorization with Support Vector Machines: Learning with many relevant features,” *Proc. ECML*, pp. 137–142, 1998.
- [4] B. Riedel, I. Augenstein, G. P. Spithourakis, S. Riedel, “A simple but tough-to-beat baseline for the Fake News Challenge stance detection task,” *arXiv:1707.03264*, 2017.
- [5] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [7] I. Ahmad et al., “Detection of fake news using machine learning,” *Proc. IJCAI Workshop on NLP for Internet Freedom*, 2020.
- [8] L. Bottou, “Large-scale machine learning with stochastic gradient descent,” *Proc. COMPSTAT*, pp. 177–186, 2010.
- [9] C. D. Manning, P. Raghavan, H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.